August 2025

Graded

Student
Luca Marzullo

Total Points
13 / 30 pts

Question 1
Efficient declarative algorithms — **7.5** / 10 pts

1.1 **Amortized time complexity** — **2** / 2 pts

- ✔ **– 0 pts** Correct
- **– 0.5 pts** Small error
- **– 1 pt** Incomplete definition
- **– 2 pts** Incorrect

1.2 **Ephemeral and persistent algorithm** — **1.5** / 2 pts

- **– 0 pts** Correct
- ✔ **– 0.5 pts** Small error
- **– 1 pt** Incomplete answer
- **– 1 pt** One algorithm missing
- **– 2 pts** Incorrect

1.3 **Amortized constant-time ephemeral queue** — **2** / 3 pts

- **– 0 pts** Correct
- **– 0.5 pts** Minor errors
- ✔ **– 1 pt** Some errors
- **– 2 pts** Many errors
- **– 3 pts** Incorrect

1.4 **Worst-case constant-time ephemeral queue** — **2** / 3 pts

- **– 0 pts** Correct
- ✔ **– 1 pt** Some errors
- **– 2 pts** Many errors
- **– 2.5 pts** Initialization correct only
- **– 3 pts** Incorrect

Question 2
One_for_all supervision — **2** / 2 pts

- ✔ **– 0 pts** Correct
- **– 0.5 pts** Error in definition
- **– 1 pt** Missing definition
- **– 0.5 pts** Missing scenario
- **– 3 pts** Incorrect

Question 3
FoldL and port objects — **0.5** / 3 pts

- **– 0 pts** Correct
- **– 0.5 pts** Minor error in FoldL
- ✔ **– 1 pt** Errors in FoldL
- **– 1.5 pts** Missing FoldL definition
- **– 0.5 pts** Minor error in NewPortObject
- **– 1 pt** Errors in NewPortObject
- ✔ **– 1.5 pts** Missing NewPortObject definition
- **– 3 pts** Incorrect

Question 4
Erlang's generic server — **0** / 5 pts

- **– 0 pts** Correct
- **– 1 pt** Missing functionality of specific module
- **– 1 pt** Missing functionality of generic module
- **– 1 pt** Missing API of specific module
- **– 1 pt** Missing API of generic module
- **– 1 pt** Missing callback interface
- ✔ **– 5 pts** Incorrect

Question 5
Reentrant locks — **1.5** / 5 pts

- **– 0 pts** Correct
- ✔ **– 0.5 pts** Specification incomplete
- **– 1 pt** Specification missing
- **– 1 pt** Motivation missing
- **– 0.5 pts** Small error in implementation
- **– 1 pt** Errors in implementation
- ✔ **– 2 pts** Implementation missing
- **– 0.5 pts** Token passing incomplete
- ✔ **– 1 pt** Token passing missing
- **– 5 pts** Incorrect

Question 6
Safety and liveness — **1.5** / 5 pts

- **– 0 pts** Correct
- ✔ **– 0.5 pts** Execution definition wrong
- ✔ **– 0.5 pts** Prefix definition wrong
- ✔ **– 0.5 pts** Extension definition wrong
- ✔ **– 1 pt** Safety definition wrong
- ✔ **– 1 pt** Liveness definition wrong
- **– 1 pt** Safety example wrong
- **– 1 pt** Liveness example wrong
- **– 5 pts** Incorrect

Please write all answers inside the rectangles

First and last name	LUCA MARZULLO
NOMA	94952100

Final exam LINFO1131

August 14, 2025
Prof. Peter Van Roy

Q1: Efficient declarative algorithms [10pts] ***(Q1 corresponds to the midterm)***

Q1.1: Define the concept of *amortized time complexity*. Assume big-O notation is known. [2pts]

Prenons le cas où l'on a n opérations qui ont toute comme complexité $O(f(n))$. La Amortized time complexity ~~n'est~~ est de $O(f(n)/n)$

cela permet de prévoir le temps que prendront les opérations à l'effectuer

Q1.2: Define the concepts of an *ephemeral algorithm* and a *persistent algorithm*. [2pts]

Prenons une Queue Q

ephemeral

Dans un algo ephemeral, la Queue sera changée sans que l'on puisse récupérer ses anciens états

persistent

Dans un algo persistent, la Queue ne sera pas changée et l'on aura accès à ses anciens états

First and last name	CUCA MARZULLO
NOMA	99851100

Q1.3: Give the code of an amortized constant-time ephemeral queue in sequential functional programming (no single assignment and no lazy evaluation). [3pts]

```
fun {NewQueue} q(nil nil) end
fun {Check Q}
   case Q of q(nil R) then q({Reverse R} nil) else Q end
end
fun {Insert Q X}
   case Q of q(F R) then q(F X|{Check R})
   end
end
fun {Delete Q X}
   case Q of q(F R) then F1 in q(F1 {Check X|R})
   end
end
```

Q1.4: Give the code of a worst-case constant-time ephemeral queue in sequential functional programming with single assignment (no lazy evaluation). [3pts]

```
fun {NewQueue} X in q(X X) end

fun {Insert Q X}
   case Q of q(S E) then E1 in E=X|E1 q(S E1) else Q end
end

fun {Delete Q X}
   case Q of q(S E) then S1 in S=X|S1 q(S1 E) else Q end
end
```

First and last name	LUCA MARZUCLO
NOMA	44952100

Q2: One_for_all supervision [2pts]
Explain how one_for_all supervision works in Erlang and give a practical scenario of its use. As part of your answer, define the concept of "supervisor tree".

un supervisor tree est composé de noeuds enfants et d'un noeud racine. chaque parent contrôle les actions de ses enfants.

dans la stratégie one for all, tous les enfants collaborent entre eux, et lorsque l'un d'entre eux a une erreur et doit redémarrer, tous les enfants le doivent aussi.

Q3: FoldL and port objects [3 pts]
Give the Oz code for the FoldL operation and use it to define a port object.

```
fun {FoldL X Acc}
   case X of nil then Acc
   [] H|T then X|{FoldL H F Acc}
   end
end
```

P = {FoldL 5}

un port nous permet de faire du message passing

First and last name	LUCA MARZUCCO
NOMA	49852100

Q4: Erlang's generic server [5pts]

To make it easier to build concurrent and fault-tolerant applications, Erlang provides a set of generic components called "behaviors". One of the most-used Erlang behaviors is the generic server. In an application using a generic server, there will be two modules, a specific module and a generic module. For this question, give a simple example of an application that uses the generic server (show it as a block diagram). Answer the following questions. What is the functionality of the specific module (what are its operations)? What is the functionality of the generic module (what are its operations)? What does the external interface of the specific module look like? What does the external interface of the generic module look like? How are the specific and generic modules connected together (what programming technique is used)?

First and last name	LUCA MARZULLO
NOMA	94852100

Q5: Reentrant locks [5pts]

For this question, answer the following four points:

1. Specify a reentrant lock.
2. Explain why reentrancy is important. Give a scenario where it is needed.
3. Give an implementation of the reentrant lock, similar to what we saw in the course.
4. Explain how the implementation works by showing how it does token passing.

un reentrant lock est un lock qui permet à un thread de retourner dans ce lock plusieurs fois d'affilée (dans la section critique)

c'est important dans le cas où un thread doit s'exécuter plusieurs fois et écrire ses information avant les autres.

First and last name	LUCA MARZULLO
NOMA	49852100

Q6: Safety and liveness [5pts]

For this question, answer the following four points:

1. Give formal definitions of execution, prefix of an execution, and extension of a prefix.
2. Give a formal definition of a safety property.
3. Give a formal definition of a liveness property.
4. Give an example of a safety property and an example of a liveness property.

une execution ϵ est dite safety si pour quand un préfixe $F =$ False, alors l'extension de F = False aussi

Une exécution ϵ est dite liveness si quand un préfixe F=True alors l'extension de F= True aussi

exemple safety : une exécution sera faite au plus une fois

liveness : une exécution sera faite au moins une fois

en gros safety assure qu'il n'y a pas d'erreur et liveness que le programme avance